DESARROLLO WEB CLIENTE - 2.º DAW

UD 02: Javascript y React

JAVASCRIPT

Material de estudio completo: sintaxis, funciones y objetos predefinidos

Curso 2026-2027

Elaborado por: José Fernández González

Versión 1.3 · octubre de 2026

CC BY-NC-ND 4.0

https://github.com/jstechg/material-docente/tree/main/2DAW_Desarrollo_Web_Cliente

Índice de estudio

1. Punto de partida: entorno y conceptos básicos
2. Ejecución de JavaScript en el navegador y modo estricto
3. Declaraciones, constantes y alcance
4. Elevación de declaraciones (hoisting)
5. Tipos de datos, null y paso de valores
6. Conversiones de tipo y template literals
7. Operadores y expresiones
8. Control del flujo
9. Definición y uso de funciones
10. Alcance léxico, contexto de ejecución y pila de llamadas
11. Argumentos variables y pseudoarray arguments
12. Objetos JavaScript, JSON y acceso seguro
13. Expansión de datos con spread
14. Extracción de datos mediante desestructuración
15. Utilidades y objetos integrados del lenguaje
16. Trabajo avanzado con arrays
17. Organización del código con módulos: import / export
18. Objetos del navegador, Window, temporización y ventanas
19. Almacenamiento básico en el navegador
20. Resumen comparativo para el examen

CÓMO ESTUDIARLO

Cada apartado incluye: 1) memoriza la idea clave; 2) ejecuta el ejemplo; 3) intenta predecir la salida antes de verlo en consola. Los ejemplos están pensados como recordatorios cortos, no como programas completos.

1. Punto de partida: entorno y conceptos básicos

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Base HTML y CSS	Para trabajar con esta unidad conviene dominar la estructura básica de una página HTML y la aplicación elemental de CSS mediante etiquetas, clases e identificadores.	Hola + .aviso { font-weight: bold; }
ECMAScript y JavaScript moderno	ES6 (ECMAScript 2015) constituye una referencia importante del JavaScript moderno; el estándar ECMAScript continúa evolucionando mediante revisiones periódicas.	let mensaje = "Hola";
Lenguaje interpretado	El código se ejecuta sin una fase de compilación tradicional previa; los errores aparecen al ejecutar o gracias a herramientas del editor.	console.log("Se ejecuta en el navegador");
Tipado débil / blando	Las variables no quedan asociadas de forma rígida a un tipo y JavaScript puede realizar conversiones entre tipos.	let x = 5; x = "cinco";
Objetos y funciones	Las funciones son valores/objetos que pueden guardarse, pasarse y ejecutarse. El lenguaje usa objetos de forma intensiva.	const saludar = function(){ return "Hola"; };
Prototipos	El modelo de objetos de JavaScript se apoya en prototipos, aunque la sintaxis moderna permita trabajar también con class.	const a = []; console.log(Array.isArray(a));

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Cliente, servidor y escritorio	JavaScript se usa en navegador y también fuera de él (por ejemplo NodeJS).	node fichero.js // ejecución fuera del navegador
Convención lowerCamelCase	En este material se utilizará la convención lowerCamelCase para nombrar variables y funciones.	let nombreCompleto; function calcularMedia() {}
Front-end / back-end	El front-end ejecuta la interfaz y la interacción en el navegador. El back-end se ocupa de la lógica del servidor, la persistencia, la autenticación y las APIs.	Front-end: valida y muestra un pedido. Back-end: guarda el pedido en una base de datos.
DOM	El Document Object Model representa la página como una estructura de nodos que JavaScript puede consultar y modificar.	document.querySelector("#saludo").textContent = "Buenos días";
Gestor de paquetes	Herramienta que instala y administra dependencias de un proyecto. En el entorno del curso se utiliza npm.	npm install
Patrón de diseño	Solución organizativa reutilizable para problemas que aparecen repetidamente en el desarrollo. No es código concreto, sino una forma de estructurarlo.	Ejemplo: separar la lógica de acceso a datos de la lógica de presentación.
Renderizar	Proceso por el que el navegador representa visualmente en pantalla el resultado de HTML, CSS y los cambios realizados por JavaScript.	Al añadir un al DOM, el navegador lo muestra en la lista visible.
TypeScript (contexto)	Lenguaje basado en JavaScript que añade tipado estático y se transforma a JavaScript para poder ejecutarse en los entornos habituales.	let unidades: number = 3; // sintaxis TypeScript
Entorno de desarrollo	Un entorno moderno combina editor/IDE, depuración, control de versiones y gestión de dependencias.	VS Code + Git + npm
Estándares web / W3C	Los estándares web favorecen que HTML, CSS y las APIs del navegador se comporten de forma interoperable entre plataformas y navegadores.	Ejemplo: una misma etiqueta <button> conserva su significado básico en navegadores compatibles.

2. Ejecución de JavaScript en el navegador y modo estricto

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Código dentro de HTML	Se puede escribir JavaScript directamente dentro de <script>.	<script>console.log("Hola");</script>
Fichero JavaScript externo	En los ejemplos del módulo se prioriza enlazar un fichero .js externo para separar estructura y comportamiento.	<script src="app.js"></script>

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Manejador de eventos	También puede incluirse JavaScript en atributos de eventos, aunque no es el método habitual del curso.	<button onclick="saludar()">Saludar</button>
Pseudo-protocolo javascript:	También existe la sintaxis javascript: para invocar código desde un enlace, aunque no es la opción habitual en código moderno.	Validar
Ubicación de <script>	Si el código necesita que el DOM esté cargado, conviene colocar el script antes de cerrar </body>.	... <script src="app.js"></script> </body>
Strict mode	"use strict" restringe comportamientos problemáticos y ayuda a detectar errores. Se recomienda usarlo.	"use strict"; let edad = 20;
Consola del navegador	Herramienta de depuración integrada en el navegador. Permite ejecutar JavaScript y observar valores o mensajes durante el desarrollo.	console.log("Valor actual:", 27);
Modificar contenido del DOM	JavaScript puede cambiar el contenido textual de un elemento ya existente sin recargar toda la página.	document.querySelector("#estado").textContent = "Listo";
Modificar atributos	Los atributos de un elemento HTML pueden consultarse o cambiarse desde JavaScript.	document.querySelector("#foto").setAttribute("alt", "Vista del producto");
Modificar estilos CSS	La propiedad style permite cambiar estilos en línea desde JavaScript. Para proyectos grandes suele preferirse alternar clases CSS.	document.querySelector("#estado").style.fontWeight = "bold";
innerHTML	Permite insertar o sustituir contenido HTML dentro de un elemento. Debe usarse con cuidado cuando el contenido procede del usuario.	document.querySelector("#resultado").innerHTML = "Completado";
document.write()	Escribe directamente en el flujo del documento. Su uso moderno es desaconsejado porque, si se ejecuta tras la carga, puede reemplazar el contenido de la página.	document.write(" Demostración ");
Listener / addEventListener()	Un listener espera un evento y ejecuta una función cuando éste ocurre. addEventListener() permite asociar manejadores sin escribirlos como atributos HTML.	document.querySelector("#guardar").addEventListener("click", () => console.log("Guardado"));

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Eventos habituales	Entre los eventos frecuentes están blur, change, focus, select, submit, keydown, keyup, click, dblclick, mouseover, mouseout, mousedown, mouseup, mousemove, load, error y resize. El evento unload existe históricamente, pero hoy se evita en código nuevo.	window.addEventListener("resize", () => console.log(window.innerWidth));
Crear elementos HTML	document.createElement() crea un nodo nuevo. Después debe incorporarse al documento para que aparezca en la página.	const li=document.createElement("li"); li.textContent="Vite";
Insertar un elemento	append() o appendChild() permiten incorporar nodos a un elemento contenedor. append() admite también texto.	document.querySelector("#lista").append(li);
Eliminar un elemento	remove() elimina del DOM el elemento sobre el que se invoca.	document.querySelector("#aviso").remove();

RECUERDA

En los ejemplos y prácticas del módulo se utilizará "use strict"; cuando corresponda trabajar en modo estricto.

3. Declaraciones, constantes y alcance

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
var	Tiene ámbito de función. Si se declara fuera de funciones, su alcance es global. No respeta el bloque { } como let.	if (true) { var x = 1; } console.log(x); // 1
let	Tiene ámbito de bloque: sólo existe dentro del bloque { } donde se declara.	if (true) { let y = 2; } // console.log(y); -> error
Variable no declarada	Sin strict mode, una asignación a un identificador no declarado puede crear una variable global. Con strict mode, da error.	"use strict"; // z = 3; -> error
const	Tiene ámbito de bloque como let y no permite reasignar la referencia/valor.	const PI = 3.1416; // PI = 3; -> error
const con arrays/objetos	No se puede reasignar la variable, pero sí modificar el contenido del objeto/array al que apunta.	const a=[1,2]; a[0]=9; // permitido // a=[]; -> error
JavaScript es case-sensitive	Distingue mayúsculas y minúsculas en identificadores y palabras reservadas.	let total = 1; let Total = 2; // son variables distintas
Identificadores	Un identificador puede comenzar por una letra, _ o $. Después puede contener también dígitos. No puede coincidir con una palabra reservada.	let _contador = 0; let $elemento = null;
Comentarios	Los comentarios de una línea usan // y los de bloque se delimitan con /* y */.	// comentario breve /* comentario de varias líneas */
Separador decimal	En el código JavaScript los números decimales se escriben con punto, no con coma.	const notaMedia = 7.5;
Punto y coma	JavaScript dispone de inserción automática de punto y coma, pero escribirlo explícitamente evita ambigüedades y mantiene un estilo uniforme.	const curso = "2DAW";

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Palabras reservadas	Términos como function, return, switch, try o catch tienen significado propio en el lenguaje y no deben usarse como nombres de variables.	const functionValue = 1; // válido; "function" sola no sería un identificador válido
Constantes simbólicas	Es frecuente escribir en MAYÚSCULAS_CON_GUIONES las constantes que representan valores fijos de configuración. Es una convención, no una obligación del lenguaje.	const MAX_INTENTOS = 3;

4. Elevación de declaraciones (hoisting)

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO

Hoisting de var	La declaración se considera disponible antes de su posición textual, pero la asignación no. Por eso el valor previo es undefined.	"use strict";
		console.log(total); // undefined
		var total = 12;

Hoisting de funciones declaración	Una declaración de función puede invocarse antes de aparecer textualmente en el fichero porque queda disponible al crear el contexto de ejecución.	mostrarEstado();
		function mostrarEstado(){
		console.log("Preparado"); }

Expresiones de función	La variable que contiene una función expresión no puede usarse como función antes de que se le asigne la función.	// calcular(); -> error const calcular = function(){ return 6 * 7; };

5. Tipos de datos, null y paso de valores

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
number	Representa números enteros y reales.	let edad=23; let pi=3.14;
boolean	Representa valores lógicos true y false y participa en conversiones desde otros tipos.	let activo = true;
string	Cadena de caracteres delimitada por comillas simples o dobles; también puede escribirse con template literals.	let nombre = "Ana";
undefined	Valor típico de una variable declarada sin valor.	let dato; console.log(dato); // undefined
object	Los valores no primitivos (arrays, objetos, fechas, etc.) se tratan como objetos.	let producto={nombre:"Teclado"};
function	typeof identifica las funciones como function.	function f(){}; console.log(typeof f); // function
typeof	Devuelve una cadena con el tipo del operando.	console.log(typeof 12.75); // "number"
typeof null	typeof null devuelve "object"; para comprobar específicamente si un valor es null conviene compararlo de forma estricta con null.	let v=null; console.log(v === null); // true
Primitivos: copia por valor	Al pasarlos a una función se copia su valor; modificar el parámetro no cambia la variable original.	function reajusta(n){ n=99; } let n=7; reajusta(n); // n sigue 7
Objetos: referencia compartida	Al pasar un objeto, una modificación de sus propiedades dentro de la función se observa fuera.	function actualizar(o){ o.precio=19.95; } const producto={precio:12.50}; actualizar(producto);

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
instanceof	Comprueba si un objeto pertenece a la cadena de prototipos asociada a un constructor.	const fecha = new Date(); console.log(fecha instanceof Date); // true
null: ausencia intencional	null suele usarse cuando se quiere indicar explícitamente que en ese momento no hay un objeto o valor asignado.	let usuarioSeleccionado = null;

6. Conversiones de tipo y template literals

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Coerción implícita	JavaScript puede convertir automáticamente un valor a otro tipo durante una operación.	const codigo="7", unidades=3; console.log(codigo+unidades); // "73"
Conversión explícita	El programador fuerza la conversión mediante funciones o métodos.	let n=42; let s=n.toString();
parseInt / parseFloat	Convierten cadenas a número entero o real.	parseInt("15",10); // 15 parseFloat("3.14"); // 3.14
== / !=	Comparan permitiendo coerción de tipos. En código nuevo suele preferirse la comparación estricta.	5 == "5" // true
=== / !==	Comparación estricta: compara valor y tipo, sin coerción. Uso recomendado.	5 === "5" // false

Coerción a boolean	Entre los valores falsy se encuentran undefined, null, 0, la cadena vacía y NaN; otros valores habituales se convierten en true.	Boolean("") // false
		Boolean(1) // true

Template literals	Cadenas entre acentos graves que permiten insertar expresiones con ${...}.	let nombre="Ana"; console.log(`Hola ${nombre}`);
String(valor)	Convierte explícitamente un valor a cadena.	String(42); // "42"
Number(valor)	Convierte explícitamente un valor a número cuando la conversión es posible.	Number("12.5"); // 12.5

7. Operadores y expresiones

7.1. Asignación

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
=	Asigna el valor de la derecha a la variable de la izquierda.	let x = 5;
+=	Suma y reasigna.	x += 2; // x = x + 2
-=	Resta y reasigna.	x -= 1;
***=**	Multiplica y reasigna.	x *= 3;
/=	Divide y reasigna.	x /= 2;
%=	Calcula el resto y reasigna (aparece en los ejemplos).	x %= 2;

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
+ / += con cadenas	El operador + concatena cadenas y += permite añadir texto al valor existente de una variable.	let etiqueta = "UT"; etiqueta += "02"; // "UT02"

7.2. Aritméticos

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
+ - * / %	Operaciones aritméticas básicas y resto de división.	let r=(5+3)*2; let resto=7%3; // 1

++x / x++	Preincremento modifica antes de usar el valor; postincremento usa primero y modifica después.	let x=5; let a=++x; // x=6,a=6
		let y=5; let b=y++; // b=5,y=6

--x / x--	Funcionan de forma análoga para decrementar.	let x=5; let a=--x; // 4
-x	Negación unaria: obtiene el valor numérico negativo sin cambiar x.	let x=5; let y=-x; // -5

7.3. Comparación y lógicos

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
=== / !==	Igualdad y desigualdad estrictas.	5 !== "5" // true
== / !=	Igualdad y desigualdad con conversión de tipos.	5 == "5" // true
> < >= <=	Comparan orden/magnitud.	edad >= 18
&&	AND: true si ambos operandos son verdaderos.	true && false // false
||	OR: true si al menos uno es verdadero.	false || true // true
!	NOT: invierte el valor lógico.	!true // false
Ternario ? :	Alternativa compacta: condición ? valorSiTrue : valorSiFalse.	let tipo = edad>=18 ? "adulto" : "menor";
Nullish ??	Devuelve alternativa sólo si el valor es null o undefined.	let telefono; console.log(telefono ?? "Sin teléfono");
Precedencia y paréntesis	Los operadores no se evalúan todos con la misma prioridad. Si el orden de evaluación es importante o puede resultar ambiguo, usa paréntesis para hacerlo explícito.	const total=(4+2)*3; // 18; sin paréntesis: 10

8. Control del flujo

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
if / else	Ejecuta un bloque u otro según una condición. Como criterio de estilo se mantienen las llaves incluso cuando el bloque contiene una sola instrucción.	if (edad >= 18) { console.log("Mayor"); } else { console.log("Menor"); }
for	Bucle con inicialización, condición y actualización del índice.	for (let i=2; i<=10; i+=2) { console.log(i); }
while	Repite mientras la condición sea true; puede no ejecutarse nunca.	let i=0; while(i<3){ console.log(i); i++; }
do...while	Comprueba la condición al final, por lo que el cuerpo se ejecuta al menos una vez.	let i=0; do { console.log(i); i++; } while(i<3);

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
break	Interrumpe el bucle inmediatamente.	for(let i=0;i<10;i++){ if(i===3) break; }
continue	Salta la iteración actual y continúa con la siguiente.	for(let i=0;i<5;i++){ if(i===2) continue; console.log(i); }
else if	Permite encadenar varias condiciones antes de llegar al else final.	if (nota >= 9) { console.log("Sobresaliente"); } else if (nota >= 5) { console.log("Aprobado"); } else { console.log("Suspenso"); }
switch / case / default	Selecciona entre varios casos a partir de un valor. break evita continuar ejecutando los casos siguientes y default cubre los valores no contemplados.	const turno = "tarde"; switch (turno) { case "mañana": console.log("M"); break; case "tarde": console.log("T"); break; default: console.log("Otro"); }
try / catch	Permite capturar una excepción y ejecutar una alternativa en lugar de interrumpir el programa de forma no controlada.	try { JSON.parse("{mal}"); } catch (error) { console.log("JSON no válido"); }
Bucles anidados	Un bucle puede contener otro. Es útil, por ejemplo, para recorrer estructuras de dos dimensiones o generar patrones.	for (let fila = 0; fila < 2; fila++) { for (let col = 0; col < 3; col++) console.log(fila, col); }
for...in	Recorre las claves enumerables de un objeto. Resulta útil para inspeccionar propiedades; en arrays suele ser preferible for...of o los métodos de Array.	const equipo={cpu:"Ryzen",ram:32}; for (const clave in equipo) console.log(clave,equipo[clave]);
for...of	Recorre directamente los valores de un iterable como un array o una cadena.	for (const nota of [7,8,9]) console.log(nota);
finally	Bloque opcional asociado a try/catch que se ejecuta al finalizar, haya ocurrido o no una excepción.	try { procesar(); } catch (e) { console.error(e); } finally { console.log("fin"); }
throw	Permite generar explícitamente una excepción cuando una condición invalida la continuación normal del programa.	if (edad < 0) throw new Error("Edad no válida");

PELIGRO TÍPICO

En while/do-while debe existir una condición que pueda dejar de cumplirse; de lo contrario se produce un bucle infinito.

9. Definición y uso de funciones

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Funciones como valores	Pueden guardarse en variables, objetos y arrays; pasarse como argumentos y devolverse desde otras funciones.	function operar(a,b,fn){ return fn(a,b); }
Parámetros sin tipo	Los parámetros no declaran tipo y JavaScript no comprueba que el número de argumentos coincida.	function f(a,b){}; f(1); // b queda undefined
Parámetros por defecto	Se puede asignar un valor en la declaración para usarlo si no se pasa argumento.	function potencia(base, exp=2){ return base ** exp; }

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
return	Devuelve un valor. Si no hay return, la función devuelve undefined.	function sinReturn(){}; console.log(sinReturn()); // undefined
Declaración de función	Tiene nombre y su declaración participa en hoisting.	function suma(a,b){ return a+b; }
Expresión de función	Una función puede asignarse a una variable; la expresión termina con ;.	const suma = function(a,b){ return a+b; };
IIFE	Una expresión de función puede invocarse inmediatamente si sólo se necesita una vez.	(function(){ console.log("Una vez"); })();
Función flecha	Sintaxis compacta muy usada como callback; no tiene su propio this ni arguments y no se usa como constructor.	const suma = (a,b) => a+b;
Callback como parámetro	Una función puede recibir otra función y ejecutarla.	function aplicar(a,b,fn){ return fn(a,b); } aplicar(6,2,(x,y)=>x*y); // 12

9.1. this en función normal y flecha

```
const dispositivo = {
  estado: "activo",
  mostrarEstado: function () {
    console.log(`Estado: ${this.estado}`);
  }
};
dispositivo.mostrarEstado(); // Estado: activo

const dispositivo2 = {
  estado: "activo",
  mostrarEstado: () => console.log(this.estado)
};
// La función flecha no enlaza un this propio con el objeto que la contiene.
```

RECUERDA PARA EXAMEN

Declaración: queda disponible durante la creación del contexto. Expresión: sólo puede usarse después de inicializar la variable. Flecha: sintaxis compacta, sin this ni arguments propios y no utilizable como constructor.

10. Alcance léxico, contexto de ejecución y pila de llamadas

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Ámbito de función (var)	Una var declarada dentro de una función sólo existe en esa función, aunque no quede limitada por bloques internos.	function f(){ if(true){ var x=1; } console.log(x); }
Ámbito de bloque (let/const)	let y const sólo existen en el bloque { } donde se declaran.	if(true){ let y=2; } // y no existe fuera
Contexto de ejecución	Cada llamada a una función crea un nuevo contexto de ejecución y su ámbito.	function saluda(){ let nombre="Ana"; }
Scope chain / lexical scope	Si una variable no está en el ámbito actual, se busca en ámbitos exteriores según dónde está escrito el código.	let a="A"; function f(){ let b="B"; function g(){ console.log(a,b); } g(); }

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Call stack	Es la pila de llamadas activas: indica qué funciones se han llamado, no qué variables son visibles.	global → primera() → segunda() → tercera()
Diferencia clave	Que tercera() sea llamada desde segunda() no le da acceso a variables de segunda() si tercera() está declarada fuera de ella.	function segunda(){ let b=2; tercera(); } function tercera(){ /* no ve b */ }

11. Argumentos variables y pseudoarray arguments

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
arguments	En funciones no flecha permite acceder a todos los argumentos recibidos, incluso los no asociados a parámetros. Tiene length e indexación, pero no todos los métodos de Array.	function producto(){ let r=1; for(let i=0;i<arguments.length;i++){ r*=arguments[i]; } return r; } producto(2,3,4); // 24
Limitación en flechas	Las funciones flecha no disponen de su propio arguments.	const f = () => { /* arguments no pertenece a esta flecha */ };

12. Objetos JavaScript, JSON y acceso seguro

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Objeto inicial	Se representa con llaves sin propiedades.	const configuracion = {};
Propiedades y valores	Un objeto agrupa propiedades separadas por comas; pueden contener objetos o arrays anidados.	const articulo={codigo:"P42", ubicacion:{pasillo:3}, etiquetas:["oferta"]};
Acceso con punto	Se accede a una propiedad mediante objeto.propiedad.	articulo.codigo
Acceso con corchetes	También se accede mediante objeto["propiedad"].	articulo["ubicacion"]["pasillo"]
Añadir propiedades	Se pueden crear nuevas claves mediante asignación.	articulo.stock = 18;
Métodos en objetos	Una propiedad puede contener una función y usar this para acceder al propio objeto.	articulo.resumen=function(){ return `${this.codigo}: ${this.stock}` ; };
Función creadora de objetos	Una función puede devolver un nuevo objeto con datos y métodos.	function crearTarea(titulo){ return {titulo, completada:false, marcar(){this.completada=true;}}; }
JSON.stringify	Convierte un valor/objeto a una cadena con formato JSON.	const texto = JSON.stringify({id:17, activo:true});
JSON.parse	Reconstruye un valor/objeto a partir de una cadena JSON válida.	const datos = JSON.parse(texto);
Optional chaining ?.	Accede a una propiedad sólo si la parte anterior existe; evita encadenar comprobaciones manuales.	const correo = pedido.cliente?.contacto?.email;
Array.isArray()	Distingue de forma fiable si un valor es un array, algo especialmente útil al recorrer propiedades de objetos heterogéneos.	Array.isArray([1,2]); // true
Object.keys()	Devuelve un array con las claves propias enumerables de un objeto.	Object.keys({id:7,nombre:"Eva"}); // ["id","nombre"]
Object.values()	Devuelve un array con los valores propios enumerables.	Object.values({a:2,b:4}); // [2,4]

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Object.entries()	Devuelve pares [clave, valor] y facilita recorrer objetos con for...of.	for (const [k,v] of Object.entries({x:1,y:2})) console.log(k,v);

DISTINCIÓN ÚTIL AL ESTUDIAR

Conviene distinguir dos conceptos relacionados: el objeto JavaScript que existe en memoria y su representación textual en formato JSON. JSON.stringify() serializa datos y JSON.parse() reconstruye un valor a partir de una cadena JSON válida.

13. Expansión de datos con spread (...)

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Copiar arrays	Expande los elementos de un array dentro de otro nuevo array.	const a=[1,2]; const copia=[...a];
Concatenar arrays	Permite combinar varios arrays de forma compacta.	const total=[...[1,2], ...[3,4]];
Combinar propiedades de objetos	Expande propiedades de un objeto y permite añadir o sobrescribir otras.	const configNueva={...config, tema:"oscuro"};
Pasar argumentos	Expande un array como argumentos de una función.	const nums=[1,2,3]; const suma=(a,b,c)=>a+b+c; suma(...nums); // 6

14. Extracción de datos mediante desestructuración

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Objetos	Extrae propiedades del objeto a variables con el mismo nombre.	const {codigo, estado} = pedido;
Renombrar	Permite asignar una propiedad a una variable con otro nombre.	const {codigo: idPedido} = pedido;
Valor por defecto	Si la propiedad no existe, se puede proporcionar un valor por defecto.	const {prioridad="normal"} = pedido;
Arrays	La asignación es posicional.	const [origen, destino] = ["ALC","VLC","MAD"];
Resto en arrays	...resto captura el resto de elementos.	const [cabecera, ...datos] = ["ID",17,23,31]; // datos=[17,23,31]

15. Utilidades y objetos integrados del lenguaje

15.1. Tratamiento numérico y eval

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
toFixed(dígitos)	Devuelve la representación redondeada con el número de decimales indicado.	(3.14159265).toFixed(2); // "3.14"
parseInt	Convierte a entero.	parseInt("15",10); // 15
parseFloat	Convierte a número real.	parseFloat("3.14"); // 3.14

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
NaN	Valor especial que indica un resultado numérico no válido.	0/0 // NaN
isNaN(valor)	Permite comprobar si un valor no es un número válido.	isNaN("tres") // true
eval(cadena)	Interpreta una cadena como código JavaScript. Debe evitarse en código ordinario porque ejecutar texto como código introduce riesgos de seguridad y dificulta el mantenimiento.	eval("2+3"); // 5
toString([base])	Convierte un número a cadena. Opcionalmente puede indicar la base de representación.	(255).toString(16); // "ff"
toPrecision(dígitos)	Devuelve una cadena con el número total de cifras significativas indicado; puede usar notación exponencial si resulta necesario.	(123.456).toPrecision(4); // "123.5"

15.2. String

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
length	Número de caracteres de la cadena.	"Hola".length // 4
toLowerCase() / toUpperCase()	Devuelven una nueva cadena en minúsculas/mayúsculas. No modifican la original.	"Hola".toUpperCase(); // "HOLA"
concat(cadena)	Devuelve una cadena concatenada.	"Hola ".concat("Ana"); // "Hola Ana"
charAt(posición)	Devuelve el carácter de una posición; se empieza en 0.	"Hola".charAt(1); // "o"
indexOf(texto[,inicio])	Busca la primera aparición desde el inicio indicado.	"banana".indexOf("na"); // 2
lastIndexOf(texto[,inicio])	Busca la última aparición.	"banana".lastIndexOf("na"); // 4
replace(texto1,texto2)	Devuelve una cadena con una sustitución.	"Hola Ana".replace("Ana","Luis");
split(separador[,límite])	Divide la cadena y devuelve un array.	"a:b:c".split(":"); // ["a","b","c"]
substring(inicio[,fin])	Devuelve una subcadena.	"JavaScript".substring(0,4); // "Java"
Secuencias de escape	Permiten representar caracteres especiales dentro de una cadena. \n crea un salto de línea, \t una tabulación y \" permite incluir comillas dobles.	console.log("Línea 1\n\tLínea 2");

15.3. Date

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
new Date()	Crea la fecha/hora actual.	const ahora = new Date();
Fecha concreta	Constructor con año, mes (0-11), día y opcionalmente hora/minuto/segundo.	new Date(2032, 6, 14, 9, 45, 0);
getFullYear()	Devuelve el año con cuatro dígitos.	fecha.getFullYear();
getMonth()	Devuelve el mes entre 0 y 11.	fecha.getMonth();
getDate()	Devuelve el día del mes.	fecha.getDate();
Modificar fecha	Puede combinarse getDate()/setDate() para avanzar o retroceder días.	fecha.setDate(fecha.getDate()+1);

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Comparar fechas	< y > permiten ordenar cronológicamente fechas; para comprobar si dos objetos Date representan el mismo instante se pueden comparar sus valores getTime().	fecha1.getTime() === fecha2.getTime();

toDateString() / toUTCString()	toDateString() genera una representación legible de la fecha local; toUTCString() devuelve una representación textual basada en UTC.	const f = new Date(2031, 3, 8);
		f.toDateString();
		f.toUTCString();

getDay()	Devuelve el día de la semana con valores de 0 (domingo) a 6 (sábado).	const f = new Date(2031, 3, 8); console.log(f.getDay());
getHours() / getMinutes() / getSeconds()	Extraen hora, minutos y segundos de un objeto Date usando la hora local.	const ahora = new Date(); console.log(ahora.getHours(), ahora.getMinutes(), ahora.getSeconds());
getTime()	Devuelve el número de milisegundos transcurridos desde el 1 de enero de 1970 UTC. Es útil para comparar instantes o calcular diferencias.	const inicio = new Date(2030, 0, 1); console.log(inicio.getTime());

15.4. Math

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Math.pow(base,exp)	Potencia.	Math.pow(2,3); // 8
Math.sqrt(num)	Raíz cuadrada.	Math.sqrt(9); // 3
Math.round(num)	Redondeo al entero más cercano.	Math.round(3.6); // 4
Math.ceil(num)	Redondeo hacia arriba.	Math.ceil(3.1); // 4
Math.floor(num)	Redondeo hacia abajo.	Math.floor(3.9); // 3
Math.max(...)	Mayor de los argumentos.	Math.max(3,9,4); // 9
Math.min(...)	Menor de los argumentos.	Math.min(3,9,4); // 3
Math.random()	Número pseudoaleatorio entre 0 (incluido) y 1 (no incluido).	Math.random();
Math.PI	Constante π.	Math.PI;

16. Trabajo avanzado con arrays

16.1. Creación, acceso y métodos básicos

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Crear arrays	Pueden crearse con new Array() o con notación literal [ ]. Pueden contener tipos distintos.	const datos=[11,"hola",true];
Índices y length	El primer índice es 0 y length indica la longitud.	datos[0]; datos.length;
Huecos / undefined	Si se accede a una posición sin dato, se obtiene undefined.	const a=[]; a[3]="x"; console.log(a[1]); // undefined
String.split → Array	split() es una forma habitual de obtener arrays a partir de cadenas.	"sol luna mar".split(" ");
join([sep])	Une elementos en una cadena.	["a","b"].join("-"); // "a-b"

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
push(elemento)	Añade al final y modifica el array.	const a=[1]; a.push(2); // [1,2]
pop()	Quita el último elemento del array y devuelve ese valor.	const a=[1,2]; a.pop(); // 2; a=[1]
reverse()	Invierte el array original.	[1,2,3].reverse(); // [3,2,1]
sort()	Ordena el array original. Sin una función comparadora, los elementos se comparan a partir de su representación como cadenas.	["c","a","b"].sort(); // ["a","b","c"]
slice(inicio[,fin])	Devuelve una porción sin necesidad de tomar todo el array.	[0,1,2,3].slice(1,3); // [1,2]
find(callback)	Devuelve la primera coincidencia que cumple una condición.	[3,8,10].find(n=>n>5); // 8
shift()	Elimina el primer elemento del array y devuelve ese valor; el resto de elementos se reindexa.	const cola = ["A", "B", "C"]; const primero = cola.shift(); // "A"

delete array[indice]	El operador delete puede eliminar la propiedad correspondiente a una posición, pero deja un hueco y no reajusta length. No suele ser la mejor opción para quitar elementos de un array.	const datos = ["A", "B", "C"];
		delete datos[1];
		console.log(datos.length); // 3

Arrays multidimensionales	Un array puede contener otros arrays. Para acceder a un valor se utiliza un índice por cada nivel.	const matriz = [[10, 20], [30, 40]]; console.log(matriz[1][0]); // 30

16.2. Recorrer arrays con callbacks

CALLBACK

En estos métodos la callback puede recibir, en este orden: valor, índice y array. Normalmente basta con el valor.

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
forEach(callback)	Ejecuta la callback para cada elemento. No construye un nuevo array de resultados; su retorno útil es undefined.	["A","B"].forEach((v,i)=>console.log(i,v));
map(callback)	Ejecuta la callback y devuelve un nuevo array con cada valor retornado.	[1,2,3].map(n=>n*5); // [5,10,15]
every(callback)	true si todos los elementos cumplen la condición.	[2,4,6].every(n=>n%2===0); // true
some(callback)	true si al menos un elemento cumple la condición.	[1,3,4].some(n=>n%2===0); // true
filter(callback)	Crea otro array formado sólo por los elementos para los que la callback devuelve true.	[1,2,3,4].filter(n=>n%2===0); // [2,4]
reduce(callback)	Reduce todos los elementos a un único valor; usa un acumulador.	[1,3,5].reduce((acc,n)=>acc+n); // 9

16.3. forEach frente a map

```
const valores = [2, 4, 6, 8];
const r1 = valores.forEach((valor) => valor / 2);
console.log(r1); // undefined

const r2 = valores.map((valor) => valor / 2);
console.log(r2); // [1, 2, 3, 4]
```

16.4. Pasar de función declarada a anónima y flecha

```
const datosMixtos = ["azul", "verde", 17, "rojo"];
function esTexto(dato) {
  return typeof dato === "string";
}
console.log(datosMixtos.every(esTexto));
console.log(datosMixtos.every(function (dato) {
  return typeof dato === "string";
}));
console.log(datosMixtos.every((dato) => typeof dato === "string"));
```

16.5. Puente hacia React: no modificar directamente

- Como preparación para React, conviene acostumbrarse a crear nuevos valores en lugar de modificar directamente los datos recibidos.
- Para añadir elementos se puede crear un array nuevo con spread.
- Para eliminar elementos se usa habitualmente filter().
- Para transformar o actualizar elementos se usa map().

```
const nombres = ["Ana", "Luis"];
const añadidos = [...nombres, "Eva"];
const sinEva = añadidos.filter((n) => n !== "Eva");
const actualizados = nombres.map((n) => n === "Luis" ? "Luis M." : n);
```

17. Organización del código con módulos: import / export

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
script type="module"	El HTML carga el fichero JavaScript principal como módulo.	<script type="module" src="principal.js"></script>
Importar un elemento	Importa una exportación concreta.	import { celsiusAFahrenheit } from "./utilidades/temperatura.js";
Importar varios	Se listan entre llaves.	import { celsiusAFahrenheit, fahrenheitACelsius } from "./utilidades/temperatura.js";
Alias al importar	as permite usar un nombre local distinto.	import { celsiusAFahrenheit as aFahrenheit } from "./utilidades/temperatura.js";
Importar todo	* as crea un objeto con las exportaciones.	import * as Temperatura from "./utilidades/temperatura.js";
export { ... }	Exporta elementos ya declarados.	export { celsiusAFahrenheit, fahrenheitACelsius };
Alias al exportar	Permite exportar con otro nombre.	export { celsiusAFahrenheit as convertirAFahrenheit };
export function	Exporta directamente al declarar.	export function esBajoCero(celsius){ return celsius < 0; }

```
// principal.js
import { celsiusAFahrenheit, esBajoCero } from "./utilidades/temperatura.js";
console.log(celsiusAFahrenheit(20));
console.log(esBajoCero(-3));

// utilidades/temperatura.js
function celsiusAFahrenheit(celsius) { return celsius * 9 / 5 + 32; }
function esBajoCero(celsius) { return celsius < 0; }
```

```
export { celsiusAFahrenheit, esBajoCero };
```

18. Objetos del navegador, Window, temporización y ventanas

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
setInterval(fn, tiempo, ...args)	Repite una acción cada cierto tiempo.	const id=setInterval(()=>console.log("tic"),1500);
clearInterval(id)	Detiene un intervalo previamente creado.	clearInterval(id);
setTimeout(fn, tiempo, ...args)	Programa una acción para que se ejecute una vez tras un retardo.	setTimeout(()=>console.log("fin"),2400);
clearTimeout(id)	Cancela un timeout pendiente.	const id=setTimeout(actualizar,3000); clearTimeout(id);
alert()	Muestra una ventana informativa.	alert("Proceso terminado");
confirm()	Muestra Aceptar/Cancelar y devuelve un valor booleano.	const guardar=confirm("¿Guardar cambios?");
prompt()	Solicita un dato al usuario mediante un cuadro de diálogo.	const curso=prompt("Curso:","2º DAW");
window	Representa la ventana o contexto global del navegador y da acceso a muchas APIs del entorno.	console.log(window.innerWidth);
document	Representa el documento cargado en la ventana. Desde él se consultan, crean y modifican elementos del DOM.	document.querySelector("#estado");
location	Permite consultar la URL actual y navegar a otra dirección.	console.log(location.href);
navigator	Ofrece información y capacidades del navegador, como idioma o estado de conexión.	console.log(navigator.language);
history	Permite desplazarse por el historial de navegación de la pestaña dentro de las restricciones del navegador.	history.back();
screen	Describe características básicas de la pantalla disponible para la ventana.	console.log(screen.width, screen.height);
window.open()	Abre una ventana o pestaña y devuelve una referencia que puede utilizarse mientras el navegador lo permita. Los bloqueadores de ventanas emergentes pueden limitarlo.	const secundaria=window.open("ayuda.html","ayuda","width=600,height=400");
close()	Una referencia de ventana abierta por script puede cerrarse mediante close(), sujeto a las políticas del navegador.	if (secundaria) secundaria.close();
window.opener	En una ventana secundaria puede hacer referencia a la ventana que la abrió, si las políticas de origen y seguridad lo permiten.	console.log(window.opener);
postMessage()	Permite intercambiar mensajes entre contextos de ventana. Al recibirlos se debe comprobar event.origin antes de confiar en los datos.	secundaria?.postMessage({tipo:"estado"}, "https://ejemplo.test");

```
function registrarPulso(etiqueta) {
  console.log(`Pulso: ${etiqueta}`);
}

const idPulso = setInterval(() => {
  registrarPulso("activo");
}, 1500);
```

```
setTimeout(() => {
  clearInterval(idPulso);
}, 6000);
```

19. Almacenamiento básico en el navegador

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
Web Storage	El navegador ofrece almacenamiento clave/valor asociado al origen. En JavaScript se trabaja principalmente con localStorage y sessionStorage.	localStorage.setItem("tema","oscuro");
localStorage	Conserva los datos entre sesiones del navegador hasta que se eliminen. Las claves y valores se almacenan como cadenas.	localStorage.setItem("curso","2DAW"); const curso=localStorage.getItem("curso");
sessionStorage	Tiene una API equivalente a localStorage, pero sus datos se asocian a la sesión de una pestaña y desaparecen al cerrarla.	sessionStorage.setItem("paso","2");
removeItem() / clear()	removeItem() elimina una clave concreta; clear() borra todas las entradas del almacén para ese origen.	localStorage.removeItem("curso");
Objetos y arrays	Como Web Storage guarda cadenas, los objetos suelen serializarse con JSON.stringify() y recuperarse con JSON.parse().	localStorage.setItem("prefs",JSON.stringify({zoom:110}));
cookies (introducción)	Las cookies son pequeños datos asociados a un sitio que pueden intervenir también en peticiones HTTP. En esta unidad basta reconocerlas; su gestión se profundiza en la unidad de almacenamiento.	document.cookie = "vista=compacta; SameSite=Lax";
Origen y seguridad	El almacenamiento y la comunicación entre ventanas están condicionados por el origen del documento y por las políticas del navegador. No se deben guardar secretos sensibles en Web Storage.	location.origin // origen actual

ALCANCE EN ESTA UD
Aquí se introduce el almacenamiento del navegador para cubrir RA3. En la UT06 se trabajará de forma específica y más profunda con persistencia, recuperación y comunicación asociada.

20. Resumen comparativo para el examen

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
var / let / const	var: ámbito de función; let: ámbito de bloque; const: ámbito de bloque y sin reasignación.	if(true){ var a=1; let b=2; const c=3; }
== / ===	== permite coerción; === exige mismo valor y tipo.	5=="5" → true; 5==="5" → false
Primitivo / objeto	Primitivos: copia por valor. Objetos: se comparte la referencia al objeto.	n no cambia / obj.propiedad sí puede cambiar
Declaración / expresión	Declaración: hoisting de función. Expresión: necesita que se haya ejecutado la asignación.	function f(){} / const f=function(){};
Normal / flecha	Flecha: sintaxis compacta, sin this propio, sin arguments propio y no constructor.	(a,b)=>a+b
forEach / map	forEach recorre; map devuelve un array nuevo con los resultados.	map(n=>n*2)
every / some	every exige todos; some exige al menos uno.	a.every(test) / a.some(test)

CONCEPTO	EXPLICACIÓN / QUÉ RECORDAR	EJEMPLO
filter / reduce	filter devuelve subconjunto; reduce devuelve un único valor acumulado.	a.filter(test) / a.reduce((acc,n)=>acc+n)
spread / destructuración	spread expande/copia; destructuración extrae valores a variables.	[...a] / const [x,y]=a
call stack / scope chain	Call stack = quién llama a quién; scope chain = qué ámbitos son accesibles según dónde está escrito el código.	global→f→g / búsqueda léxica de variables
null / undefined	null se asigna explícitamente; undefined es frecuente cuando no existe/asigna valor. Para null, usar === null.	v===null
setInterval / setTimeout	Interval repite; timeout ejecuta una vez tras el retardo.	setInterval(fn,1000) / setTimeout(fn,1000)
typeof / instanceof	typeof informa del tipo general de un valor; instanceof comprueba la relación de un objeto con un constructor/prototipo.	typeof 5 // "number" new Date() instanceof Date // true
if / else if / switch	if/else if encadena condiciones; switch resulta cómodo cuando se comparan varios casos de un mismo valor.	if (x > 0) { ... } / switch (modo) { case "A": ... }
localStorage / sessionStorage	localStorage persiste entre sesiones; sessionStorage se limita a la sesión de la pestaña.	setItem / getItem / removeItem
window / document	window representa el contexto de la ventana; document representa el documento HTML cargado y su DOM.	window.innerWidth / document.querySelector(...)

ESTRATEGIA DE EXAMEN

Si una pregunta incluye código, identifica primero: 1) ámbito de cada variable; 2) tipos y posible coerción; 3) orden de ejecución/hoisting; 4) qué devuelve la función; 5) si el método modifica el original o devuelve un nuevo valor.

Bibliografía y documentación de consulta

Las siguientes referencias se incluyen como documentación de consulta y ampliación. El contenido, la estructura y los ejemplos de este material han sido redactados específicamente para este documento y no reproducen textos, figuras ni ejemplos de las fuentes citadas.

- ECMAScript Language Specification: https://tc39.es/ecma262/
- MDN Web Docs · JavaScript y Web APIs: https://developer.mozilla.org/
- WHATWG. DOM Standard. https://dom.spec.whatwg.org/